

COMPUTATIONAL SOCIAL SCIENCE · TUTORIAL

# Nostalgia on YouTube

Collecting and analysing platform data with the YouTube Data API, Google Colab and Hugging Face

Enes Abanoz · Ondokuz Mayıs University  
University of Trento — Department of Sociology and Social Research

# What we will build today

1

## Get API access

A free YouTube Data API key

2

## Census a channel

596 videos, full statistics

3

## Harvest comments

21,469 comments from one 1997 performance

4

## Detect languages

lingua — the audience is not who you expect

5

## Measure nostalgia

Dictionary vs. zero-shot transformer

### The case

## Andrea Bocelli Con Te Partirò

Performed **1997**

Uploaded **2015**

Comments **26,936**

Views **84 million**

# Why this video: three clocks

An archival re-release carries three separate timestamps. Nostalgia is a claim about distance from a past — here we can measure that distance for every single comment.

## 1997

### Performance

Piazza dei Cavalieri.  
Fixed for every comment.

## 2015

### Upload

Catalogue back-fill.  
Fixed for every comment.

## 2015–26

### Comment

This one varies —  
and that is the variable.

*A comment written in 2024 sits 27 years after the performance and 9 years after the upload. Most YouTube research collapses these into one date and loses the structure entirely.*

# 01

## The YouTube Data API

Getting a key, and understanding what it costs

# One fact governs every design choice

A free key gives 10,000 quota units per day. The endpoints are not priced equally — and that asymmetry decides what research questions are cheap.

| Endpoint            | Units | Returns                              |
|---------------------|-------|--------------------------------------|
| search.list         | 100   | ≤50 results — same price for 1 or 50 |
| videos.list         | 1     | ≤50 videos, full statistics          |
| channels.list       | 1     | ≤50 channels                         |
| playlistItems.list  | 1     | 50 playlist entries                  |
| commentThreads.list | 1     | 100 comments                         |

10,000 units

= 100 searches

— or —

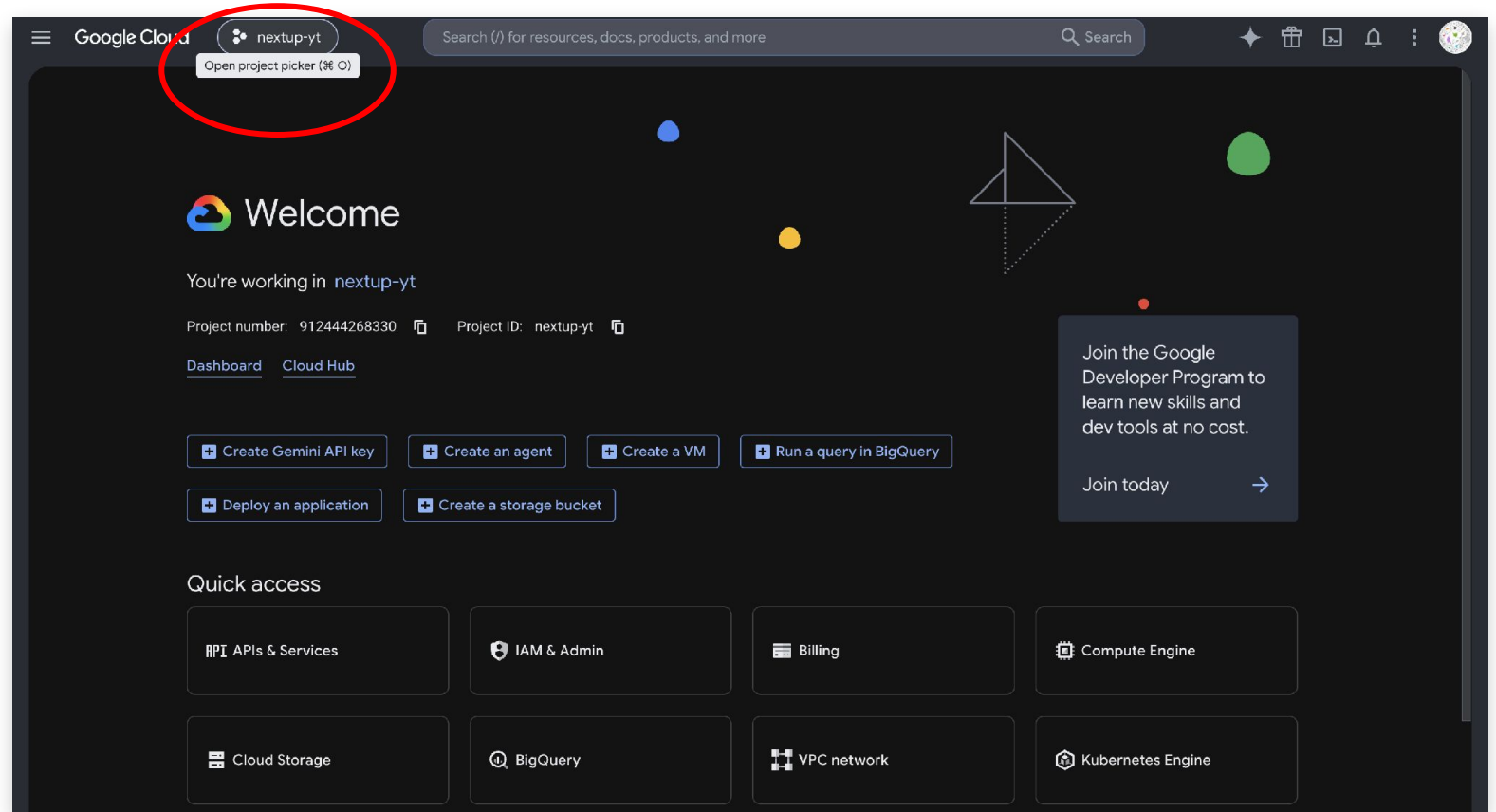
= 1,000,000 comments

Search is 100× the price of retrieval.

*Our whole 596-video channel census cost under 100 units — because it never calls search.list. Framing a question around a known channel instead of a discovery problem changes the economics completely.*

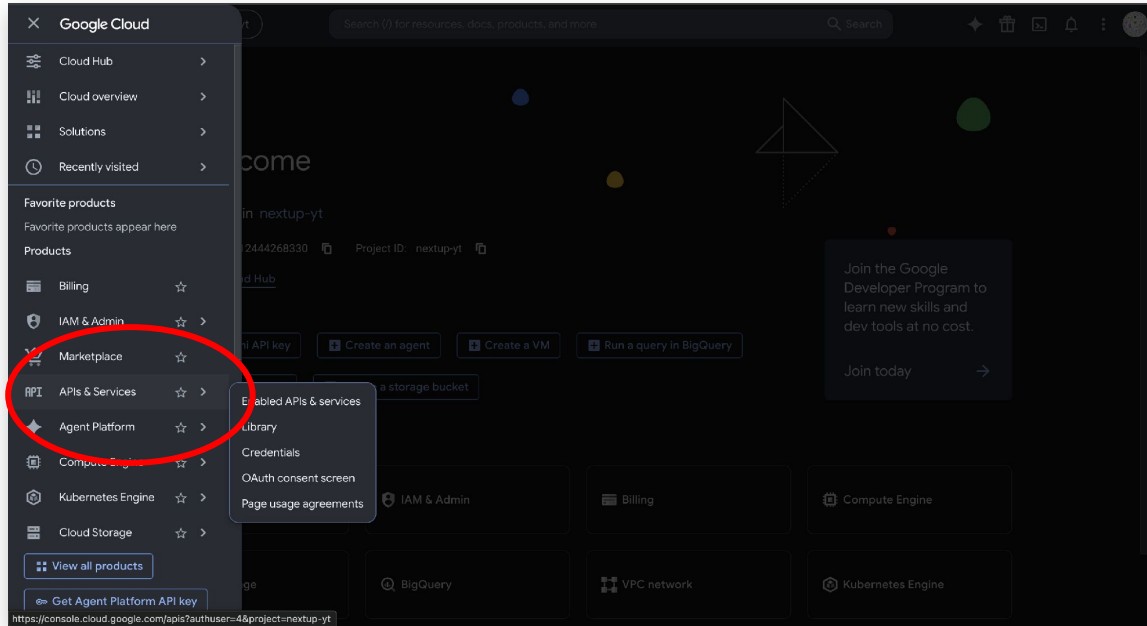
# Create a Google Cloud project

- console.cloud.google.com — sign in with any Google account
- Project dropdown in the top bar → New Project
- Name it something recognisable (here: nextup-yt)
- Wait for creation, then confirm it is the selected project

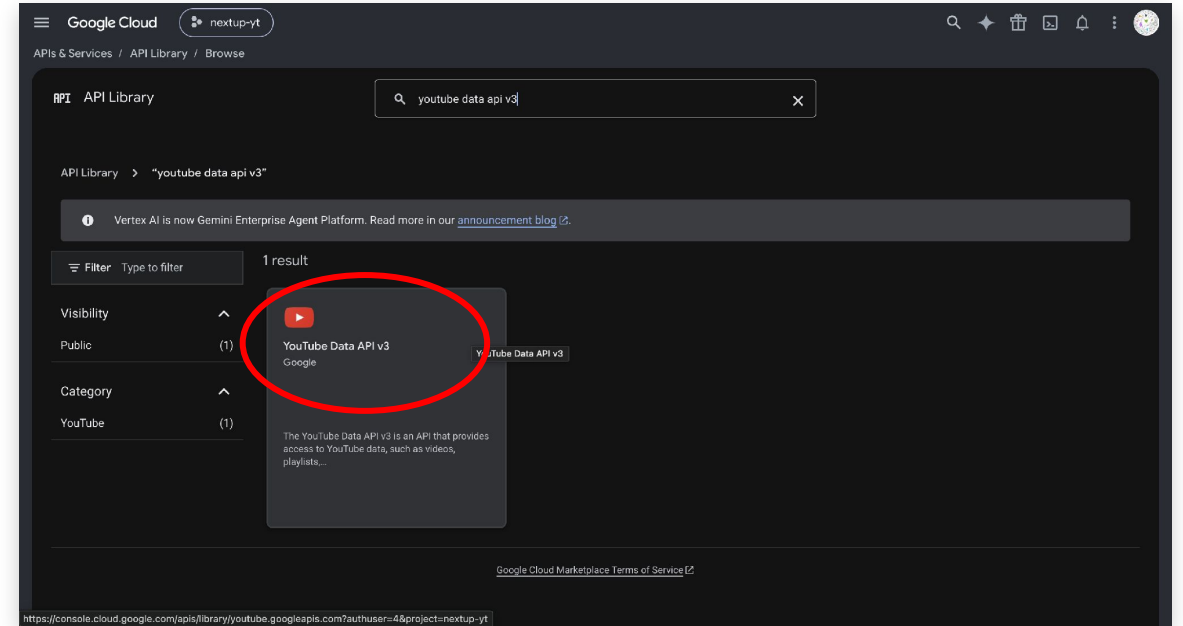


*The project picker confirms which project you are working in.*

# Find the YouTube Data API v3



Navigation menu → APIs & Services → Library



Search “youtube data api v3” — one result

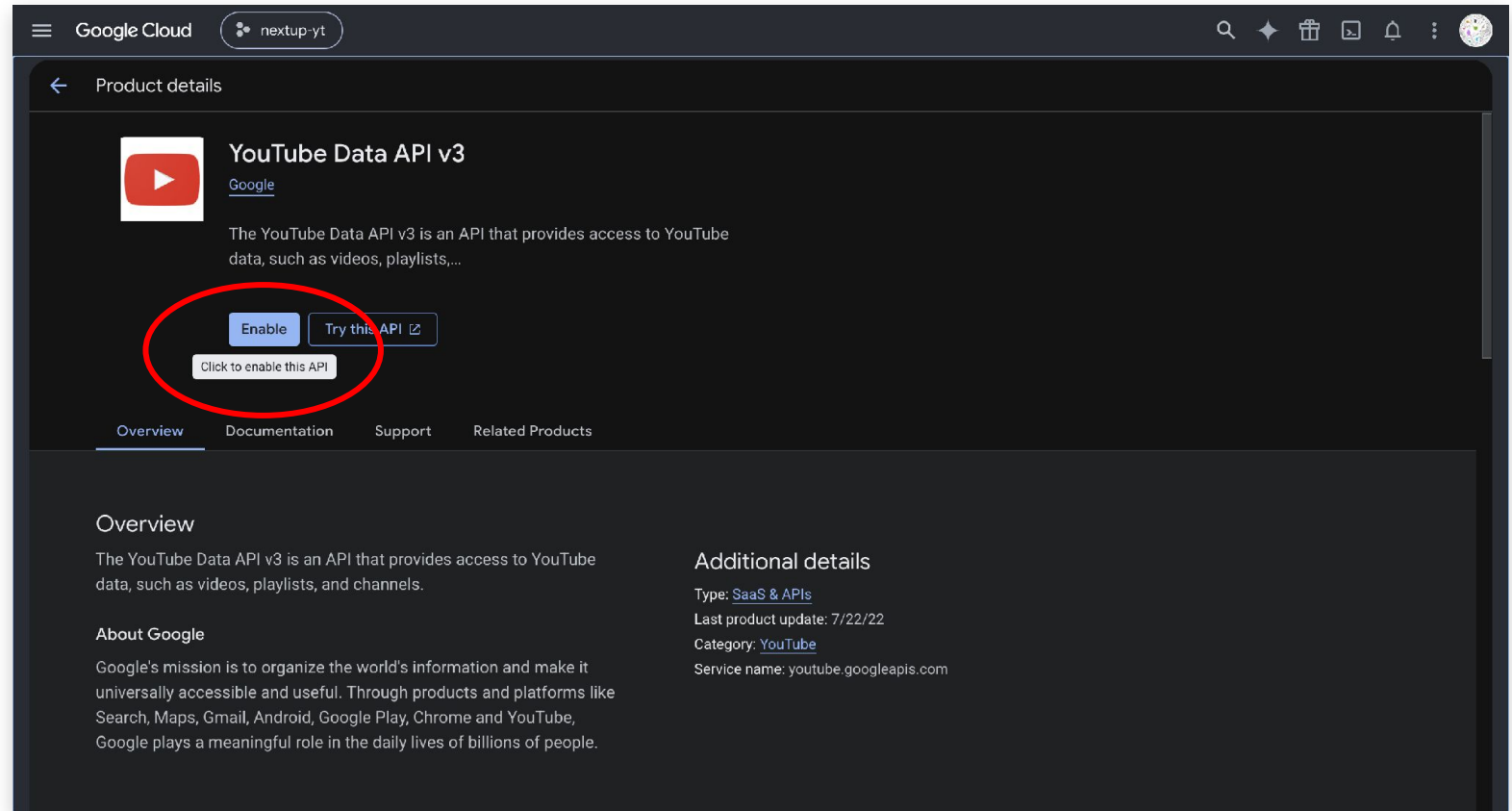
**You do NOT need OAuth.** OAuth is only for private data — your own uploads, your subscriptions, posting comments. Reading public videos and public comments needs a plain API key. If a tutorial tells you to download client\_secret.json, it is solving a different problem.

# Enable the API

- Click Enable on the product page
- Propagation takes 1–2 minutes
- A 403 accessNotConfigured error right afterwards usually just means “wait”

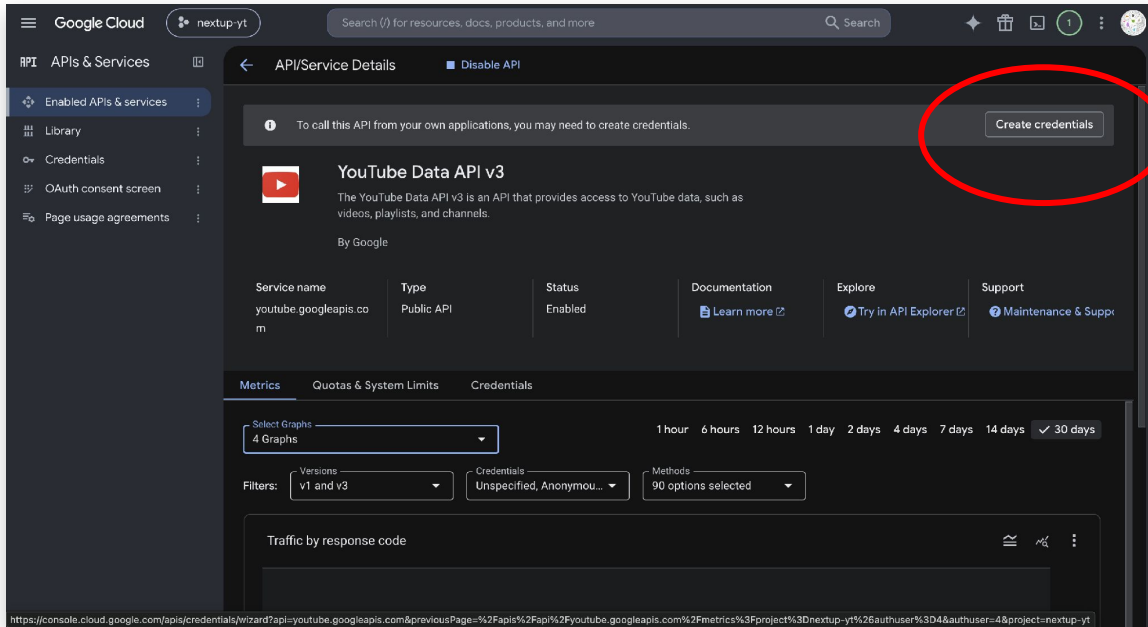
Service name

youtube.googleapis.com

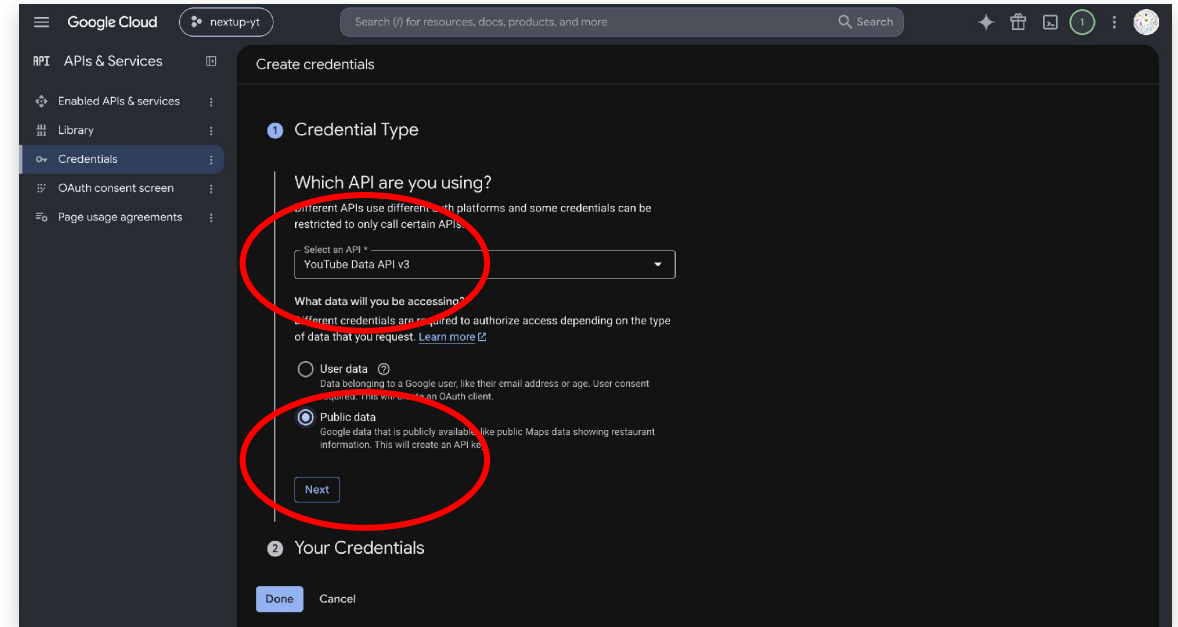


*Enabling is per-project — a second project needs its own enable.*

# Create credentials — public data



API/Service details → Create credentials

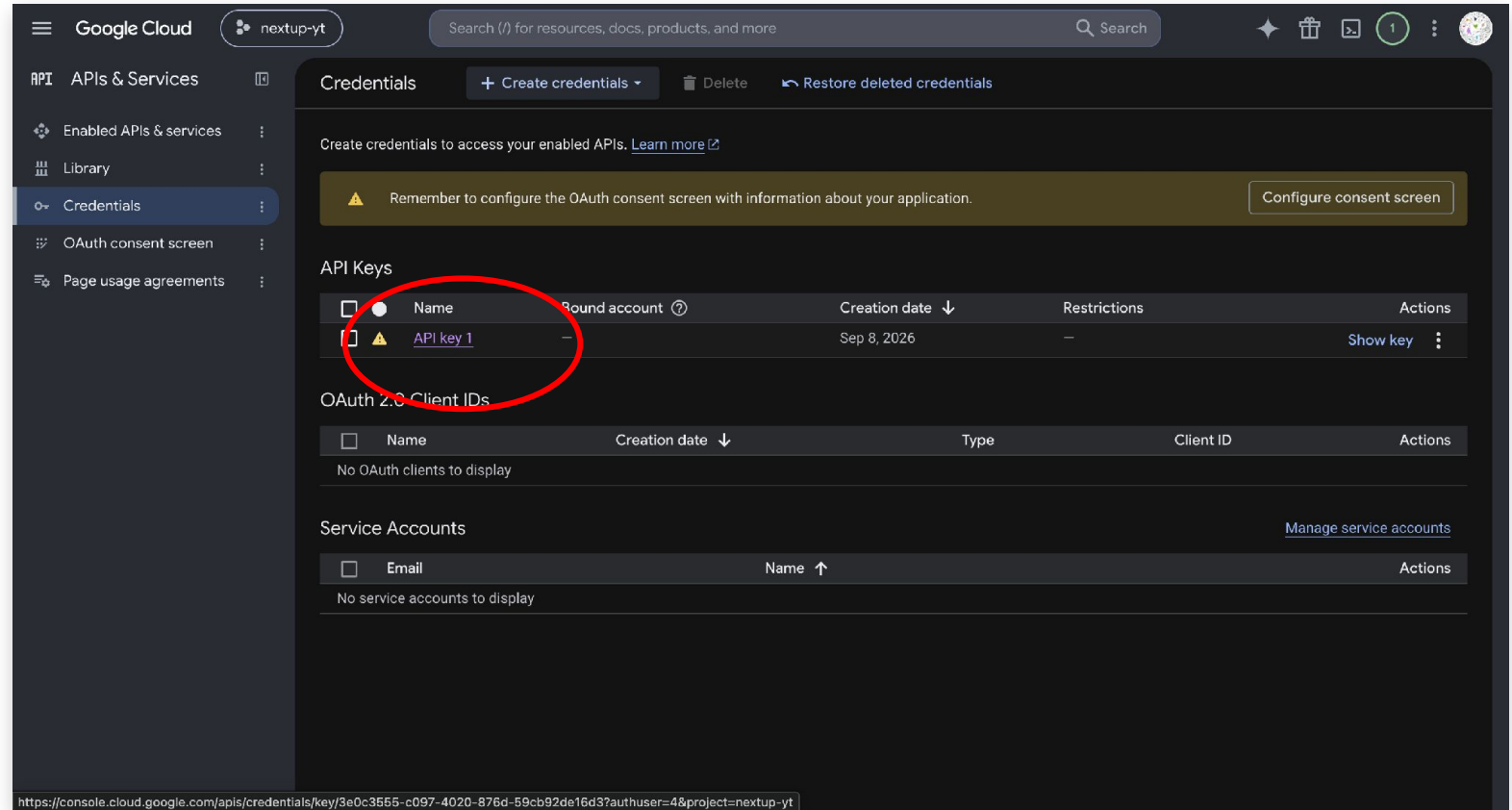


Choose “Public data” → this creates an API key

**The choice that matters.** “User data” creates an OAuth client and sends you down a consent-screen path you do not need. “Public data” gives you the plain API key that reads public videos and comments — which is the whole of this tutorial.

# Your key — and how to restrict it

- Copy the key from the Credentials page
- Click the key → API restrictions → Restrict key
- Tick only YouTube Data API v3 → Save
- A leaked restricted key can do nothing else



The warning triangle is Google reminding you the key is unrestricted.

# Never type a key into a cell

Anything typed into a notebook cell is saved inside the .ipynb file — and that file is what you push to GitHub. The leak happens at collection time; you find out much later.

## Colab Secrets

- Key icon in the left sidebar
- + Add new secret
- Name: YOUTUBE\_API\_KEY
- Toggle Notebook access on

```
from google.colab import userdata
API_KEY = userdata.get("YOUTUBE_API_KEY")

# The notebooks fall back gracefully:
#   Colab secret -> env var -> hidden prompt
# so the key never appears in the file.
```

# 02

## Colab & repository structure

Everything runs from GitHub — nothing to install

# One repository, three notebooks

```
bocelli-nostalgia/  
├── notebooks/  
│   ├── 01_channel_performance.ipynb  
│   ├── 02_harvest_and_prepare.ipynb  
│   └── 03_nostalgia_analysis.ipynb  
├── src/  
│   └── channel_client.py      API wrapper + quota meter  
├── data/  
│   ├── processed/           committed — notebooks load this  
│   └── raw/                  GITIGNORED — holds author IDs  
├── docs/  
│   ├── api_key_guide.md  
│   └── ethics_gdpr.md  
└── outputs/figures/
```

## Every notebook starts the same way

```
if "google.colab" in sys.modules:  
    subprocess.run(["git", "clone", "-q",  
                    "https://github.com/eabanoz/"  
                    "bocelli-nostalgia.git"])  
    ROOT = Path("bocelli-nostalgia").resolve()  
os.chdir(ROOT)  
sys.path.insert(0, str(ROOT))
```

- No Google Drive mount, no manual upload
- Colab preloads pandas, torch, transformers, sklearn
- src/ is a real module, imported not pasted
- Data is committed — the tutorial runs without a key

*data/raw/ is gitignored because it holds author\_channel\_id — personal data under GDPR. Pseudonymisation happens before anything leaves that folder.*

# What each notebook does

01

## Channel performance

Full census of 596 videos. Quota accounting, the age confound, choosing the target video.

API key

no GPU

02

## Harvest & prepare

21,469 comments. Pseudonymisation, cleaning, language detection, the three clocks.

optional

no GPU

03

## Nostalgia analysis

Dictionary vs. zero-shot classification. Constructs by language and over time.

no key

GPU

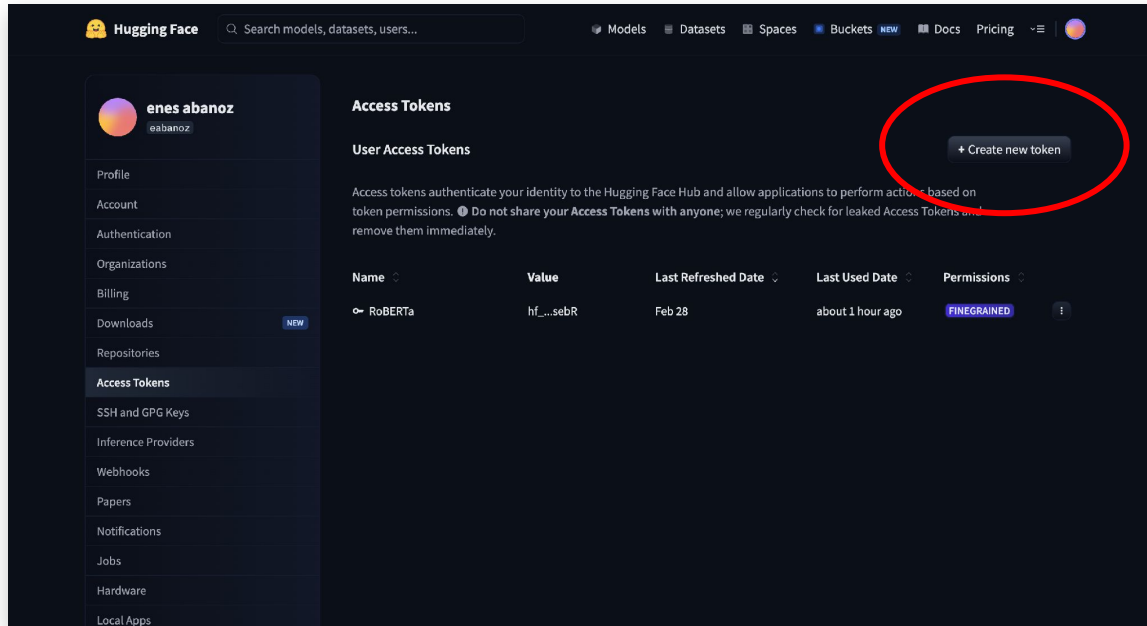
*Switches (HARVEST, SCORE) let notebooks 02 and 03 load the committed data instead of re-collecting or re-scoring.*

# 03

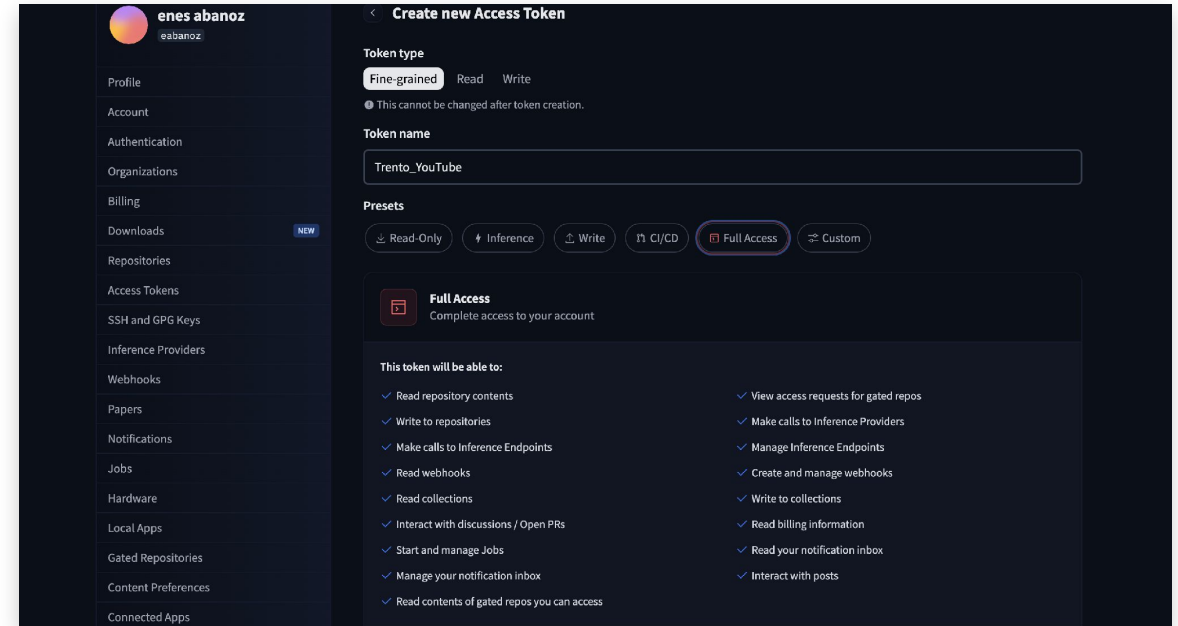
## Hugging Face

Models, tokens, and which one answers your question

# Creating a Hugging Face token



Profile → Access Tokens → Create new token



Name it, pick a preset — Read is enough for this tutorial

**Prefer Read over Full Access.** The screenshot shows Full Access selected, which grants write to repositories, billing visibility and inference management. For downloading public models you only need Read — and token type cannot be changed after creation. Store it as a Colab secret named HF\_TOKEN, never in a cell.

# Which model answers which question

## **mDeBERTa-v3-xnli**

*Zero-shot NLI*

Scores whether a comment entails a nostalgia construct. No word list, works across languages.

## **XLM-RoBERTa sentiment**

*Multilingual polarity*

Used as a contrast case — it measures the wrong thing, deliberately.

## **FEEL-IT (UmBERTo)**

*Italian sentiment*

RoBERTa architecture on Italian Common Crawl. Only 8.5% of our corpus is Italian.

**Sentiment is orthogonal to nostalgia.** Comments about dead parents classify as positive — they are affectionate in tone. Choosing the familiar tool would have measured the wrong construct.

# 04

## Research steps

From a channel ID to a measured construct

# Six decisions, not six commands

## 1 Select channel

Known channel beats a discovery problem — no search.list, no quota burn

## 2 Census uploads

596 videos. playlistItems, not search. Under 100 units

## 3 Choose one video

Comment count over view count — a nostalgia study needs text

## 4 Harvest comments

order="time", not relevance.  
21,469 retrieved = 70.3%

## 5 Detect language

lingua, restricted set, confidence retained

## 6 Measure constructs

Dictionary AND classifier.  
Compare them by language

*Every one of these is a research decision that shapes the result — and each is reversible only by re-running.*

# What counts as nostalgia

Constructs from Svetlana Boym, *The Future of Nostalgia* (2001), and adjacent work. Restorative vs. reflective is the distinction that matters most: both are nostalgia, but only the first carries a claim about how things ought to be.

## Personal memory

Autobiographical recall, usually family

## Mortality / grief

The dead, funerals, memorials

## Restorative

The past was better; a wish to return

## Reflective

Bittersweet longing, without wanting return

## Collective

Shared generational or national memory

## Anemoia

Longing for a time never personally lived

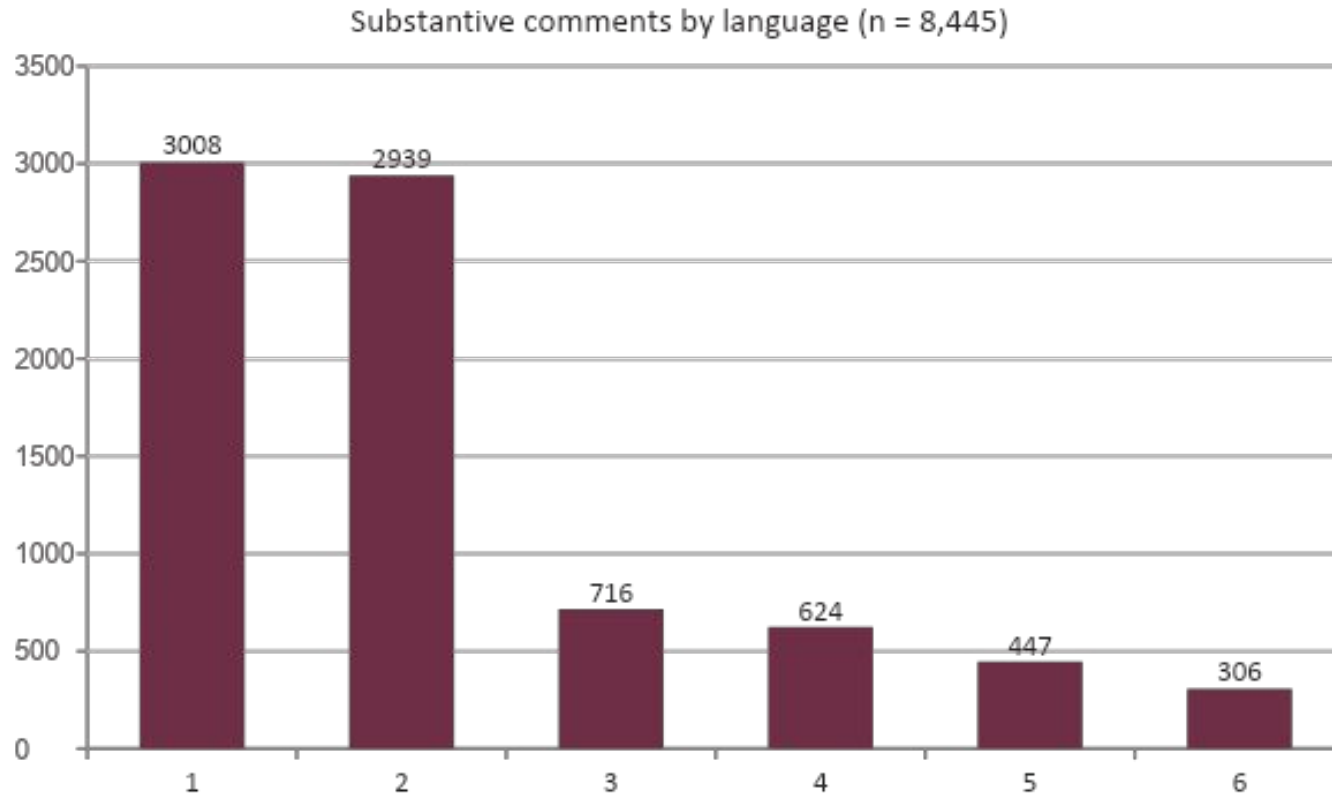
*These become hypotheses for a zero-shot classifier — the model has never seen a labelled example of any of them.*

# 05

## Analysis

Three results, and one that fell apart

# The audience is not who you would guess

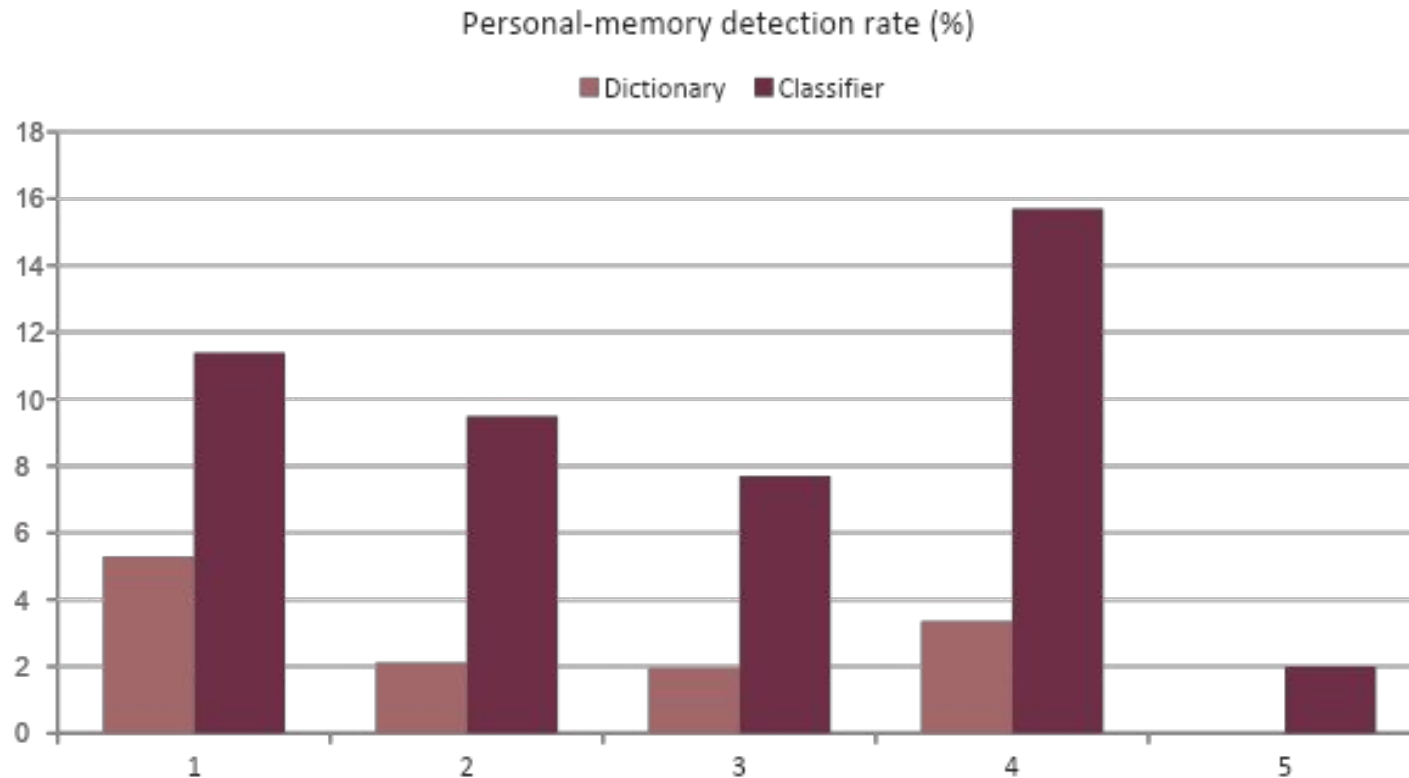


## English and Portuguese are co-dominant

- ~35% each — Bocelli's Brazilian audience is enormous
- Italian is fourth, at 8.5%
- No defensible way to call this an Italian-language corpus
- Which language 'leads' moves ten points across reasonable threshold choices

*Report the sensitivity table, not a point estimate. English has the lowest mean detection confidence of any major language — short, ambiguous Latin-script text defaults to it.*

# Dictionaries are biased toward their author's languages



## Zero

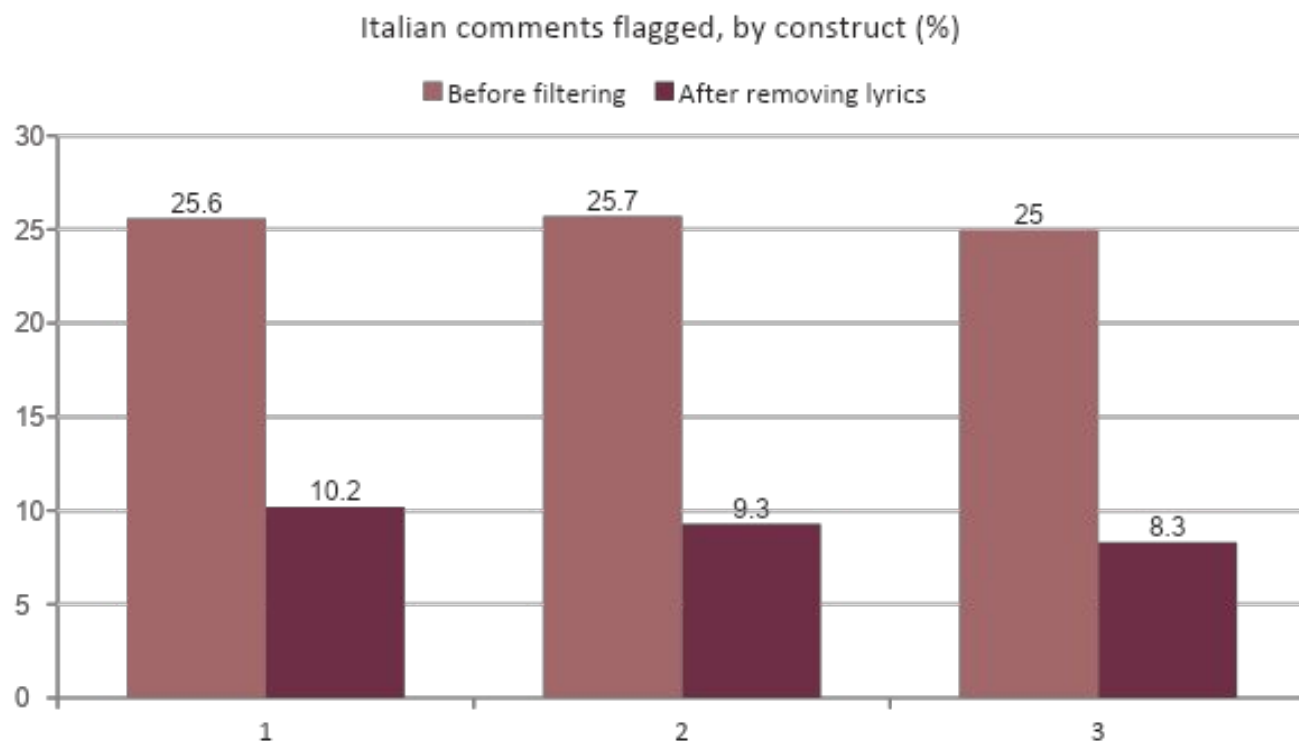
personal-memory hits across 3,585 Portuguese comments — with the first word list.

Not rarity. The list simply had no Portuguese in it. Russian is still at 0.0% — left unfixed on purpose, as an exercise.

*Every dictionary has holes shaped like the languages its author cannot read.*

*The classifier catches French “papy tu me manque” and Portuguese “meu pai queria tanto voltar à Itália” — true positives no English word list would ever find.*

# Not everything in a comment field is a comment



## The tell

Three conceptually distinct constructs all elevated in one group, by the same amount. Separate measures should not move in lockstep.

## The cause

About 21% of Italian comments are the song's lyrics, pasted repeatedly — some still carrying [Strofa 1] markers from lyrics sites. Portuguese: 0.2%.

*The song is about departure. Asked whether those lyrics concern grief and farewell, the model says yes — correctly. The classifier was never wrong. The input was not what we assumed.*

**No diagnostic warned us.** Confidence was high, the output well-formed, the story plausible and theoretically satisfying. Ten minutes reading the longest comments caught what no metric did.

# What we cannot claim

- **Not a sample of any population** No age, gender or location for any commenter. This describes discourse on one video.
- **Unvalidated** The model has never seen a labelled example of “reflective nostalgia”. These are hypothesis-generating numbers until someone hand-codes a sample and reports Cohen's  $\kappa$ .
- **One construct fails outright** “Restorative” scores below chance against its own dictionary. It should not be reported at all.
- **Length drives entailment** Longer comments score higher on every construct. Groups that write at length look more nostalgic. Report stratified.
- **70.3% retrieval** The API caps pagination below the displayed comment count. A large sample, not a census.

*A study that shows its own correction is more credible than one that presents only the surviving result.*

# Now let's run it

`github.com/eabanoz/bocelli-nostalgia`

Open notebook 01 from the README badge. You do not need an API key to follow along — the data is in the repo. Get a key if you want to collect your own.

**Runtime → Change runtime type → T4 GPU (needed for notebook 03)**